# Stock-Y

**StockY** is a next-generation financial intelligence platform built on the **MERN (MongoDB, Express, React, Node.js)** stack. In the current volatile market, retail investors are often the last to receive actionable information. StockY addresses this "Information Asymmetry" by correlating institutional "Whale Trades" (Congressional and Hedge Fund disclosures) with real-time news sentiment powered by the **Gemini 1.5 Flash AI model**.

## 1.2 The Problem Statement

Financial data is currently fragmented; investors have access to news and trade disclosures, but they lack a tool that bridges the two. Standard platforms do not explain the *intent* behind a trade. StockY solves this by using Generative AI to analyze the news cycle surrounding a specific trade, providing a "Why" behind the "What".

## 1.3 Objectives

- **Real-time Monitoring:** Tracking high-profile trades from institutional movers.
- **AI Synthesis:** Leveraging Natural Language Processing (NLP) to summarize global news into 3-sentence actionable insights.
- **User Persistence:** Providing a secure, cloud-hosted watchlist via MongoDB Atlas.
- **Workflow Automation:** Integrating CI/CD practices through automated GitHub version control.

## 2.1 The Tech Stack

StockY follows a decoupled microservices-inspired architecture:

- **Frontend (React.js):** Utilizes a state-driven approach with Tailwind CSS for a high-performance, three-panel dashboard.
- **Backend (Node.js/Express):** Handles API routing, Gemini AI integration, and middleware for data processing.
- **Database (MongoDB):** A NoSQL approach to handle flexible data structures for watchlists and AI caching.
- **AI Layer (Google Gemini):** Performs Zero-Shot sentiment analysis on news headlines to generate a "Bullish/Bearish" score.

## 2.2 Logic Flow

The system processes data through the following steps:

1. **Ingestion:** The system pulls recent trade data (Mock JSON for stability) and news headlines via External APIs.
2. **Correlation:** When a user selects a trade, the backend sends the trade details and the last 48 hours of related news to the Gemini API.
3. **Synthesis:** Gemini returns a JSON object containing a sentiment score and a logic-based summary.
4. **Caching:** The result is stored in MongoDB to optimize performance. If another user clicks the same ticker, the system serves the cached data instantly.

© FS-PEP project by Vigneshwar Balakumar

**System Design & Implementation Highlights**

## 3.1 Database Schema (Data Modeling)

The system uses two primary collections to ensure efficient data management:

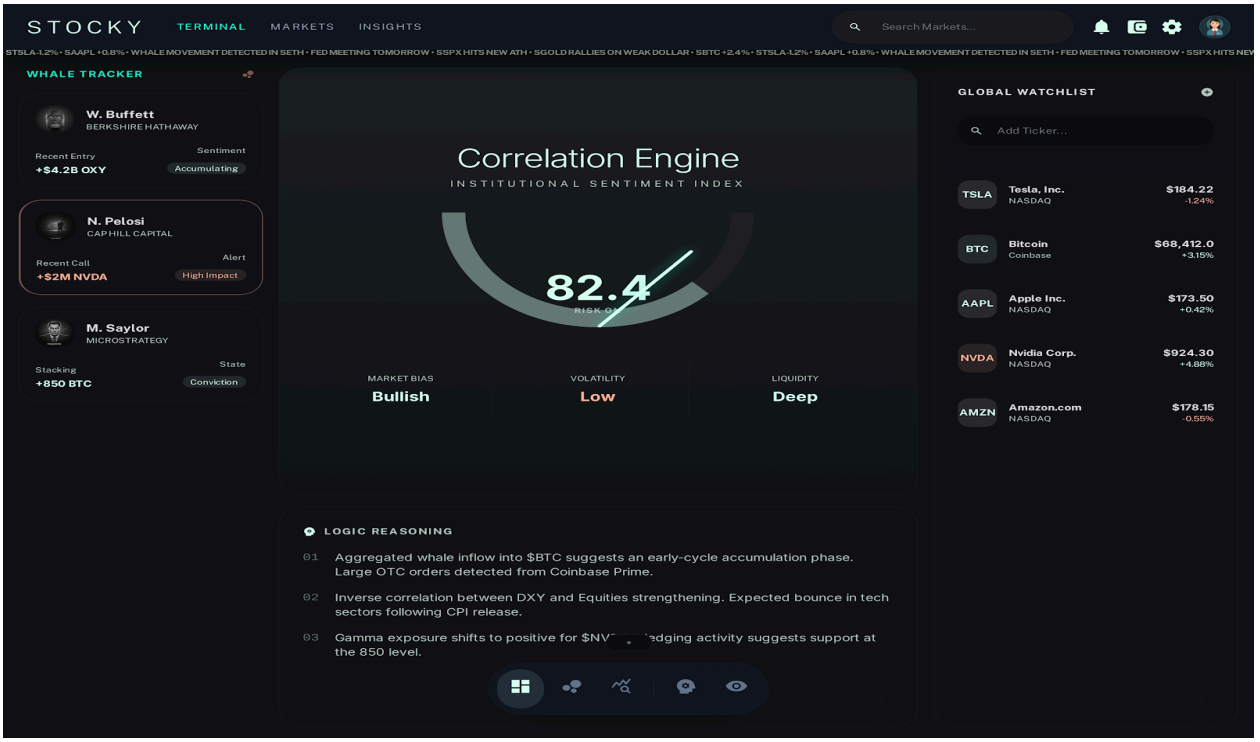
- **Watchlist Collection:** Maps User IDs to Ticker Symbols, demonstrating CRUD (Create, Read, Update, Delete) competence.
- **InsightCache Collection:** Stores `ticker`, `sentiment_score`, `summary_text`, and `timestamp`, proving the ability to optimize API costs and system latency.

## 3.2 UI/UX Design (The 3-Panel System)

The interface is divided into three functional zones:

1. **Whale Tracker (20%):** The primary data source for institutional moves.
2. **The Correlation Engine (55%):** The central processing unit where AI insights are rendered.
3. **Portfolio Manager (25%):** The user-interactive side for data persistence.

**Basic UI interface(Ai-generated)**




## 3.3 Conclusion & Future Scope

StockY represents a robust implementation of the MERN stack integrated with modern Generative AI. Beyond the current scope, the project is architected to scale toward:

- Live **SEC EDGAR API** integration.
- Real-time **WebSocket** notifications.

The inclusion of automated GitHub commits ensures that the project maintains a professional development lifecycle, ready for industrial deployment.

© FS-PEP project by Vigneshwar Balakumar